
Deep Knowledge: Days 5 & 6 (Object-Oriented Programming)

We are moving to **OOP (Object-Oriented Programming)**.

If you want to be a 10 LPA AI Engineer, this is the most important Python concept you will ever learn. AI models (like Neural Networks in PyTorch) are strictly built using OOP.

The Problem with "Normal" Coding

Until now, you probably wrote code top-to-bottom.

```
name = "Jainish"
print(name)
do_something()
```

This is called *Procedural Programming*. It is fine for 50 lines of code. But if you are building ChatGPT (millions of lines of code), top-to-bottom coding becomes an unreadable, chaotic mess.

The Solution: The Blueprint (Class) and the House (Object)

OOP is a way of organizing code. Instead of writing a list of instructions, we build **Things** (Objects) that have **Attributes** (Data) and **Behaviors** (Actions).

Let's use a Robot Factory analogy.

1. The **Class** (The Blueprint)

A **Class** is just a paper blueprint. It is a design for a robot. A blueprint cannot clean your room or talk to you. It is just paper.

2. The **Object** (The Actual Robot)

When we take that blueprint to the factory and actually build the metal robot, that robot is called an **Object**. You can build 100 different robots from the exact same blueprint.

3. The **__init__** method (The Boot-Up Sequence)

When you press the "ON" button on a brand-new robot for the very first time, it needs to set itself up. It needs to load its battery, set its name, and turn on its eyes. In Python, the

moment you create an Object, a special hidden function called `__init__` runs automatically to set it up.

4. The mystery of `self`

If I have 10 robots, and I yell "Raise your hand!", how does Robot #4 know to raise *its own* hand, and not Robot #5's hand?

In Python, `self` is the robot's way of saying "MY".

- `self.name` = MY name.
- `self.battery` = MY battery.

The 4 Essential Words of OOP

1. Class (The Blueprint)

- **What it is:** A block of code that defines a new *Data Type*. Just like Python has built-in types like `String` or `Integer`, a Class lets you create a custom type (like `AIBot` or `NeuralNetwork`).
- **Syntax:** Always starts with the word `class` and the name is always Capitalized (PascalCase).

```
class DataCleaner:
```

2. Instance / Object (The Real Thing)

- **What it is:** When you actually use the Class to create something in your computer's memory.
- **The Verb:** The act of creating an object is called "**Instantiation**" (This is a huge interview word).

```
# We are 'instantiating' the DataCleaner class to create the  
'my_cleaner' object/instance.  
my_cleaner = DataCleaner()
```

3. Attribute (The Data / Variables)

- **What it is:** These are just variables, but because they live *inside* an object, we call them Attributes. It is data that belongs to the object (like `name`, `battery`, `filepath`).

- **Syntax:** They always have `self.` attached to them.

```
self.battery = 100 # 'battery' is an attribute
```

4. Method (The Action / Functions)

- **What it is:** These are just functions (`def`), but because they live *inside* a Class, we call them Methods.
- **Syntax:** The very first parameter inside the parenthesis must **always** be `self`.

```
def clean_data(self): # 'clean_data' is a method
    print("Cleaning...")
```

How Python Actually Reads OOP (The Mechanics)

What is the deal with `__init__` and `self`? Let's look at the mechanics of what happens under the hood.

The `__init__` (Dunder Init)

- `__init__` stands for "initialize".
- Notice the double underscores on both sides? In Python, we call these "**Dunder Methods**" (Double UNDERscore).
- Dunder methods are "Magic Methods." You never actually call them yourself. Python runs them automatically in the background when a specific event happens. The `__init__` event is triggered the exact millisecond an Object is *instantiated*.

The Mechanics of `self`

Why do we have to write `self` everywhere?

When you type `bot_1.work()`, Python secretly translates that behind the scenes into:

```
AIBot.work(bot_1).
```

Python physically passes the `bot_1` object into the method so the method knows whose battery to drain. That is why the method says `def work(self):` — it is catching the `bot_1` object!
